

Lab Cheat Sheet

Exam-ready cards from classroom labs — static triage, artifacts, cred/IR, persistence, Volatility, NTFS

English · 繁體中文

[Download PDF](#) · [中文 PDF](#) · [Overview](#) · [Home](#)

Scope reminder

- Detection and forensics focus only — tool/command strings are **IOC / hunt patterns**, not attack recipes.
- Screenshots are study aids from lab notes; full lab packs / zips are not published here.

Jump to

1. Glossary
2. A. Static / sandbox commands & PE
3. B. Triage & artifacts
4. C. Cred dump / DCSync / IR response
5. D. Persistence
6. E. Memory forensics / Volatility
7. F. NTFS / MFTECmd

Glossary (compact)

Entropy / packing

High byte randomness (often >7.0) hints a file may be compressed/encrypted/packed — a lead, not proof of malware.

imphash

Hash of the PE import table order/names; similar imphash → same toolchain/family candidate for pivoting.

ssdeep

Fuzzy hash; near-matches help cluster variants that changed a few bytes.

Imports

APIs the PE claims to call (network, crypto, process). Odd imports for the claimed app type are hunt leads.

PE / MZ

Windows executable format; bytes `4D 5A` ("MZ") mark the DOS stub of a PE image in file or memory.

Process-tree lineage

Parent → child chain (e.g. `WINWORD→cmd`). Impossible parents are strong exam IOCs.

Triage

Selective copy of high-value artifacts — not a full disk image.

collection_context / log.json

Manifests that record what a collector grabbed, when, and from which host/path.

Dirty hive + .LOG1/.LOG2

Registry hive copied while OS was live; transaction logs must be collected with the hive to replay consistency.

DCSync

Abuse of directory replication APIs to pull credential material; contrast with expected DC ↔ DC replication.

krbtgt

Domain KRBtgt account; after suspected ticket abuse, rotate **twice** so old and intermediate keys both die.

Event 4104

PowerShell script-block logging; absent when the dump tool is a native EXE (no PS engine).

4688 / Sysmon 1

Process-create events that carry command-line — primary hunt surface for native dump-tool IOC strings.

Layered persistence

Multiple independent footholds; remove one and others remain.

Mutually restoring

Mechanisms that reinstall each other; need a coordinated removal window.

PID 4 boot time

System process create time $\approx$ last boot; `windows.info` alone does not give boot time.

ProfileList vs sessions

Historical logons vs who was logged in at capture time.

pslist / psscan / pstree

Active list vs pool scan (hidden/exited, DKOM unlink) vs parent/child tree.

DKOM unlink

Kernel-object manipulation that unlinks a process from the active list so `pslist` misses it.

getsids / S-1-5-18

Process token SIDs; `S-1-5-18` = local SYSTEM.

malfind

Finds private RWX regions; RWX+MZ in an odd process = lead (JIT can FP).

modules vs driverscan

Loaded images vs driver objects — compare by **address**, not name/count.

vadinfo private vs file-backed

Private = no file mapping; both hollowing and reflective load look private.

Injection / hollowing / reflective / side-load

Different memory/disk patterns — do not conflate with DLL search-order side-load.

malfind --dump vs dumpfiles

Carve private injected regions vs extract file-backed images from memory.

NTFS 0x10 vs 0x30

STANDARD_INFORMATION vs FILE_NAME timestamps; SI/FN mismatch is a timestamp **lead**.

MFTECmd

Eric Zimmerman parser for `$MFT` / USN timelines and 0x10 vs 0x30 checks.

A Static / sandbox commands & PE

01 — What each command does

WHAT EACH COMMAND DOES

- `mkdir -p output` : creates an `output/` sub-folder in the lab directory for your analysis results; `-p` suppresses an error if it already exists.
- `sha256sum ... | tee ...` : computes the **SHA-256** of the sample (your primary forensic fingerprint) and, via `tee`, both prints it to screen *and* saves it to a file.
- `file ...` : identifies the true file type by reading the binary header, regardless of the filename extension. This proves whether it really is a PE executable.
- `ls -lh ...` : shows the size in human-readable form (`-h`), plus permissions and timestamp. An unexpectedly large or small file is itself a clue.

WHAT EACH COMMAND DOES — `mkdir`, `sha256sum|tee`, `file`, `ls -lh`

What it means — Basic static triage: make a work folder, record SHA-256 (and keep a copy via `tee`), identify file type, note size.

Why it matters on exam — Exams expect a safe order: **hash first** so every later note ties to one sample identity.

Example — You see `sha256sum sample.exe | tee hash.txt` then `file sample.exe` reporting “PE32 executable” — you write the hash into the report header before sandboxing.

02 — The report's static panel

THE REPORT'S STATIC PANEL

Your workstation checks (hash, type, size, strings) are the disk-side view. When the report comes back, its

File Details / Static Analysis section parses the same PE more deeply. Read it before the behaviour:

- **Section entropy.** Entropy scores randomness from 0–8.0. A section above **~7.0** (approaching 7.9) is compressed or encrypted, the rigorous signature of **packing**. A *raw size* far smaller than the *virtual size* confirms it: the section expands in memory at run time.
- **imphash.** A hash of the PE's *import table*. Unlike the SHA-256 (which changes on a one-byte edit), the imphash clusters variants built from the same code into one family.
- **ssdeep (fuzzy hash).** Measures how *similar* two files are, grouping near-identical samples an exact hash would treat as unrelated.
- **Imports = capability.** The OS functions a PE intends to call hint at what it does before it runs: `VirtualAlloc` + `WriteProcessMemory` + `CreateRemoteThread` together advertise **process injection**. An implausibly *thin* import table is itself a packing tell.

Record the **imphash** alongside the SHA-256, so related variants group into one family instead of arriving as unrelated alerts.

What it means — **Entropy** measures randomness (packing/encryption hint). **imphash** fingerprints the import table. **ssdeep** fuzzy-matches near-duplicates. **Imports** list claimed APIs.

Why it matters on exam — Questions often ask what each field is for — and that high entropy alone ≠ malware.

Example — Panel shows entropy 7.4, imphash matching a known packer family sample, imports including `WinHttpOpen` in a “calculator” binary — hunt/cluster lead, not a conviction.

03 — PE/MZ + process-tree lineage

PE / MZ

PE (Portable Executable) is the standard file format for Windows executables, DLLs, and drivers. **MZ** refers to the two magic bytes `4D 5A` (“MZ”) at the very start of every PE file, named after Mark Zbikowski, an early MS-DOS architect. So when the report says a dropped file “carries an MZ/PE header,” it means the malware wrote another Windows executable to disk.

PROCESS-TREE LINEAGE

The **process tree** records which process started which. Malicious activity often announces itself through an anomalous parent-child relationship with no legitimate explanation. The canonical example to memorise: opening a Word document runs `WINWORD.EXE`, and that application has *no* reason to launch a shell, so `WINWORD.EXE → cmd.exe` or `→ powershell.exe` almost always means a malicious macro is now running commands. Many of the most reliable EDR rules are expressed as exactly this lineage, and they are hard to evade because the technique *requires* that parentage.

Our sample is a **standalone dropper** (no Office parent here), but the skill is the same: open each process for its **command line** (where an encoded PowerShell payload becomes visible) and its **injection targets**, then locate the **persistence**: a service, WMI subscription, or Run key, because that is both the detection and the cure.

PE/MZ + PROCESS-TREE LINEAGE (`WINWORD→cmd/powershell`)

What it means — PE files start with MZ (4D 5A). Lineage is who spawned whom in sandbox or EDR trees.

Why it matters on exam — Spot impossible parents: Office spawning `cmd / powershell` is a classic Day-1 IOC pattern.

Example — Sandbox tree: WINWORD.EXE → `cmd.exe` → encoded PowerShell — you flag the lineage and the child cmdline as evidence of macro-driven execution, not “Word is malware.”

04 — MITRE ATT&CK Day-1 subset

MITRE ATT&CK

The report's own **ATT&CK Techniques Detection** panel summarises the run as **246 indicators mapped to 102 attack techniques across 10 tactics** - a different, broader tally than the 8 · 55 · 200 indicator counts on the summary card. A Day-1 subset worth knowing:

- **T1566.001** Phishing: Spearphishing Attachment · **T1204.002** User Execution: Malicious File (the macro)
- **T1137** Office Application Startup · **T1059.001** PowerShell · **T1059.003** Windows Command Shell
- **T1027** Obfuscated Files or Information (base64 `-EncodedCommand`) · **T1105** Ingress Tool Transfer · **T1071.001** Web Protocols
- **T1543.003** Windows Service · **T1053.005** Scheduled Task

MITRE ATT&CK Day-1 subset (T1566.001 ... T1053.005)

What it means — A short technique list covering initial access → execution → persistence you'll map evidence to on exam day.

Why it matters on exam — Map artifacts to technique IDs (e.g. scheduled task ↔ T1053.005) without inventing attacker steps.

Example — Phish macro + Run key + scheduled task → cite T1566.001, T1547.001, T1053.005 with the Event IDs / paths that prove each.

B Triage & artifacts

05 — Triage = selective copy

A triage collection is a **selective copy**: a folder tree of individually collected files you browse directly - there is no volume to mount, and no unallocated space, file slack or deleted-file recovery, all of which a full disk image would give you at the cost of hours of acquisition and hundreds of gigabytes. The manifest files matter because the package **documents itself**: `collection_context.json` records what was asked for and when, `uploads.json` lists exactly which files were captured with their hashes and sizes, and `log.json` records errors and skips. Together they let you prove what is in the evidence - and what was never collected - without relying on the analyst's memory.

Triage = selective copy; `collection_context` / `uploads` / `log.json`

What it means — Triage grabs high-value paths (hive, EVTX, MFT, Prefetch...), not a full disk. `collection_context` and `log.json` document what landed.

Why it matters on exam — Know triage ≠ imaging; exam answers often hinge on reading the manifest for gaps.

Example — Manifest lists SOFTWARE hive but not SYSTEM — you note incomplete collection before concluding “no service persistence.”

06 — Artifact category → path

Artifact category	Location
Registry hives	<code>Windows\System32\config\</code>
Event logs	<code>Windows\System32\winevt\Logs\</code>
File-system metadata (<code>\$MFT</code> / <code>\$J</code>)	volume root / <code>\$Extend</code>
Prefetch	<code>Windows\Prefetch\</code>
WMI repository	<code>Windows\System32\wbem\Repository\</code>
Scheduled tasks	<code>Windows\System32\Tasks\</code>
BITS jobs database	<code>ProgramData\Microsoft\Network\Downloader\</code>
Amcache	<code>Windows\AppCompat\Programs\Amcache.hve</code>

What it means — Cheat map from evidence type to default Windows path under a triage root.

Why it matters on exam — Speed: “Where is Amcache / Prefetch / Tasks?” is pure path recall.

Example — Asked for program execution evidence → Prefetch + Amcache paths; asked for WMI persistence → permanent event subscription store under WBEM.

07 — Dirty SOFTWARE hive

Registry Explorer flagged the collected **SOFTWARE** hive as **dirty**.

a. Tick what “dirty” means here. (*tick one*)

- ☐ Windows was live when the hive was copied, so its transaction logs still hold uncommitted changes
- ☐ The hive was corrupted by the collection tool
- ☐ The hive contains malware

Dirty SOFTWARE hive quiz — live copy + transaction logs

What it means — A hive copied while Windows was running is **dirty** until transaction logs (.LOG1 / .LOG2) are replayed with a forensic parser.

Why it matters on exam — Correct answer is almost always: collect hive **and** logs — orphan hive alone may miss recent keys.

Example — Quiz: live SOFTWARE without logs → incomplete; with hive + LOG1/LOG2 → parsers can reconcile dirty writes.

c Cred dump / DCSync / IR response

Detection framing

Credential-access tooling below is listed only as **what Security/Sysmon show** and **what to hunt**. Treat suspicious dump-tool command lines as **IOC strings** (e.g. process name + path + target process), not procedures to run.

08 — DCSync vs legitimate replication

Directory replication is a **domain-controller-to-domain-controller** protocol: legitimate requests come from machine accounts ending in `$`, on a schedule. An interactive administrator account asking for `DS-Replication-Get-Changes-All` in a two-minute burst is not replication - it is a tool run. What it yields is categorically bigger than the LSASS dump: the dump exposed only the secrets cached on *this* host, whereas DCSync returns the password hash of **every account in the domain** - including `krbtgt`, which enables forged Kerberos golden tickets - without the attacker ever touching another machine.

DCSync vs legitimate DC ↔ DC replication; krbtgt risk

What it means — Directory replication APIs used by DCs are also abused to pull secrets (DCSync). Legitimate traffic is DC ↔ DC; suspicious is non-DC / unexpected subject pulling replication rights.

Why it matters on exam — Distinguish 4662 noise from “non-DC subject + replication GUIDs + odd 4624 source.”

Example — Hunt: Security 4662 with replication GUIDs where Subject is a workstation admin account, not a DC computer account — escalate as DCSync-class activity.

09 — No 4104 when dump is a native EXE

No 4104 script block wraps the dump. Script Block Logging is on and the timeline holds 65 script blocks from this host, but not one of them mentions `lsass`, `procdump` or a `.dmp` path - none resolves the LSASS PID or launches the dumping tool. Look back at the command line: `procdump64.exe -accepteula -ma 748 C:\Users\Public\lsass.dmp` carries a **hard-coded PID** and was executed *directly*, not through a PowerShell script that discovered the PID first. Script Block Logging only records PowerShell script text, so a direct executable launch leaves it untouched. The absence is a finding, not a gap in your search - and it is why **Security 4688 with command-line logging is the main detection source** for this incident.

No 4104 when dump is native EXE; **4688 / Sysmon 1 cmdline** is the primary detection surface (IOC patterns such as dump-tool name + LSASS target — not a how-to)

What it means — PowerShell script-block logging (4104) only sees script. A native dump EXE never enters the PS engine, so 4104 stays quiet.

Why it matters on exam — Pivot to process-create cmdline (4688 / Sysmon 1) for IOC strings naming dump tools and LSASS as target — detection patterns only.

Example — Exam trap: “No 4104 ⇒ no cred dump” is false. You hunt 4688/Sysmon 1 for process image + command-line tokens consistent with LSASS access tools.

10 — DCSync impact & remediation

Because DCSync returns the directory itself, the impact covers **every account in the domain**, not the handful cached on this host - so account-by-account triage is the wrong model. In particular `krbtgt` is exposed: its hash signs all Kerberos tickets, so an attacker can mint **golden tickets** impersonating any user, including ones created after the incident. Removing this attacker's access therefore means a **domain-wide credential reset**, with `krbtgt` reset *twice* (allowing for replication between resets), not just rebuilding host `AD`.

DCSync impact: domain-wide reset; krbtgt twice

What it means — If domain secrets were pulled, assume credential material is burned; remediation includes widespread password resets and KRBTGT rotation.

Why it matters on exam — Remember **krbtgt ×2**: one rotation leaves a previous key valid for ticket forgery windows.

Example — Answer key: isolate, reset passwords domain-wide as scoped by IR plan, rotate krbtgt twice, hunt Golden Ticket return paths.

11 — Domain compromise IR

Treat this as a **domain compromise, not a host compromise**. Immediately: isolate host `AD` while preserving it as evidence; disable or reset the `Administrator` account used for the dump and the replication; then plan a **domain-wide credential reset** - all privileged and service accounts first, with `krbtgt` reset **twice** to invalidate forged tickets. In parallel, hunt for the attacker's return path before resetting anything they could watch (the persistence mechanisms in Lab 2-3), restrict directory-replication rights to DC machine accounts only, and deploy the two detections above so a repeat is caught in minutes rather than after the fact.

Domain compromise IR: isolate AD, hunt return path, krbtgt×2

What it means — IR play: contain AD trust boundary, find how they return (persistence + identities), then key rotation.

Why it matters on exam — Order matters: contain → evidence → eradicate persistence → reset/rotate.

Example — Scenario answer: take affected DCs offline from untrusted segments, preserve logs/memory, hunt persistence, then krbtgt×2 after footholds are gone.

12 — Layered persistence

Categories beyond a Startup-folder shortcut include: **registry autostart values** (`Run` / `RunOnce` , `Winlogon` shell/userinit); **scheduled tasks**; **Windows services**; **permanent WMI event subscriptions**; **BITS jobs**; and **hijack-based** techniques such as DLL search-order or COM hijacking. An operator layers several because they are *independent* and live in different subsystems: each is listed by a different tool, so a defender who finds one and stops looking leaves the others intact. Redundancy also makes removal harder - if one mechanism re-fetches the payload, deleting the file alone achieves nothing. For the defender the lesson is that remediation requires a **complete list first**, then removal in one coordinated action.

Layered persistence categories; complete list then remove

What it means — Several independent mechanisms (Run key + service + task + WMI...). Removing one leaves others.

Why it matters on exam — Exam: “list all then remove” — incomplete lists fail cleanup questions.

Example — You document four footholds across Autoruns categories, confirm each with path+cmdline+time+account, then plan removal of all four.

13 — Mutually restoring persistence

The four mechanisms are independent and **mutually restoring**: the BITS job re-downloads the payload, and any surviving launcher runs it again, so a partial clean-up simply puts you back to the start. Removing them one at a time also **warns the attacker** - they still hold valid credentials (Lab 2-2), so a partial remediation invites them to re-establish before you finish. The discipline is therefore: *list everything first, remove it all inside one coordinated window, then verify* - and treat the stolen credentials as part of the same remediation, not a separate task.

Mutually restoring persistence; coordinated removal window

What it means — Mechanism A reinstalls B and B reinstalls A (or a watcher repairs both).

Why it matters on exam — Need a coordinated window (or offline image) so both die together.

Example — Task recreates a service binary and the service recreates the task — remove both in one change window while host is isolated.

E Memory forensics / Volatility

14 — Boot time + ProfileList vs sessions

`windows.info` does **not** report when the machine last booted. The System process (**PID 4**) is created at boot and runs until shutdown, so its **creation time** is the boot time.

```
vol -f <memory.raw> windows.info
vol -f <memory.raw> windows.pslist
```

Use `pslist` (or `pstree`) to read PID 4 / System create time.

ProfileList vs windows.sessions

They do **not** answer the same question:

Plugin / key	Answers
ProfileList (<code>HKLM\ ... \ProfileList</code>)	Every account that has ever logged in — historical record
windows.sessions	Who was logged in at the moment of capture

```
vol -f <memory.raw> windows.registry.hivelist
vol -f <memory.raw> windows.registry.printkey \
  --key "Microsoft\Windows NT\CurrentVersion\ProfileList"
vol -f <memory.raw> windows.sessions
vol -f <memory.raw> windows.pstree
```

What it means — Boot time ≈ PID 4 create time. ProfileList = historical profiles; sessions = live at dump time.

Why it matters on exam — Trick questions swap “ever logged in” with “logged in now.”

Example — ProfileList shows alice + bob; sessions shows only alice → bob is historical, not present at capture.

15 — pslist vs psscan vs pstree

Plugin	What it does
windows.pslist	Walks the active process list the way the OS does. Fast and accurate for anything running normally.
windows.psscan	Scans raw memory for process structures instead of trusting the list. Finds processes that have exited , and processes deliberately unlinked from the list to hide them.
windows.pstree	Shows the same processes as a parent/child tree . Most problems show up here, because malware often has a parent process that makes no sense.

Exam tip: `pslist` ≠ `psscan` → look for hidden or exited processes (including DKOM-style unlink). Use `pstree` to spot impossible lineages.

What it means — DKOM unlink = process still in memory but removed from the active list so `pslist` misses it; `psscan` can still find the EPROCESS.

Why it matters on exam — Diff the two lists; then use `pstree` for “weird parent.”

Example — `psscan` shows `malware.exe` PID 4242 absent from `pslist` → suspect unlink/hide; confirm with handles/netscan on that PID.

16 — cmdline, dlllist, getsids, handles

Plugin	What it shows	Why it matters
windows.cmdline	Full command line each process was launched with	Encoded or obfuscated arguments are a strong indicator
windows.dlllist	Modules (DLLs) each process has loaded	A networking DLL inside a process that has no reason to talk to the network is a well-known indicator

Plugin	What it shows	Why it matters
windows.getsids	Security identifiers (SIDs) a process holds — whose authority it runs under	A normally unprivileged program running as SYSTEM means privilege escalation happened. SID <code>S-1-5-18</code> is the built-in SYSTEM account (highest local privilege)
windows.handles	Open objects a process holds: files, registry keys, named pipes	Pipes are one way separate malware components talk to each other on the same host

What it means — Four “enrichment” plugins: args, modules, token SIDs, open objects (incl. named pipes).

Why it matters on exam — S-1-5-18 on a user app = priv-esc lead; odd `wininet / winhttp` in notepad-like process = C2 lead.

Example — `getsids` shows notepad as S-1-5-18; `handles` shows a suspicious named pipe between two PIDs — document both as hunt findings.

17 — netscan + malfind

windows.netscan

Recovers TCP and UDP endpoints, including sockets already **closed** by capture time. Shows local/remote address, port, connection state, and owning **PID**.

windows.malware.malfind

Flags memory that is **executable and writable** but **backed by no file on disk**. That combination is the signature of injected or reflectively-loaded code. The plugin prints a hexdump of the first bytes so you can see whether they begin `4D 5A` (MZ).

False positives

Some legitimate software allocates **RWX** memory genuinely — anything with a JIT compiler (browsers, .NET runtime).

Interesting hit = **RWX and MZ header and** a process that has no reason to be doing it.

Treat **malfind** as **leads to check, not conclusions to report**.

What it means — netscan = sockets (incl. closed). malfind = private RWX; MZ at start strengthens the lead; JIT = common FP.

Why it matters on exam — Never report “malware” from RWX alone — require context (process + MZ + behavior).

Example — RWX+MZ inside `spoolsv.exe` with netscan showing rare remote IP → escalate; RWX in `chrome.exe` without MZ → likely JIT, deprioritize.

18 — windows.modules vs windows.driverscan

These plugins do **not** report the same kind of thing.

Plugin	What it walks	How things are named
windows.modules	Kernel's list of loaded module images	As files — <code>ntoskrnl.exe</code> , <code>hal.dll</code> , Path under <code>\SystemRoot\system32\</code>
windows.driverscan	Memory for driver objects	In the device namespace — <code>\Driver\HTTP</code> , <code>\Driver\00000311</code>

One image can own several driver objects, and scanning finds the same object more than once.

What does / does not prove

- Comparing the **two counts** proves nothing — they count different kinds of thing.
- Comparing the **two name lists** proves nothing — `ntoskrnl.exe` and `\Driver\HTTP` are not in the same namespace; no name will match.

The comparison that means something

Map by **address**: take each driver object's **Start** from `driverscan` and find the module in `modules` whose **Base ... Base+Size** range contains it.

A driver object whose **Start falls inside no loaded module** is worth investigating — something is executing in kernel space that the module list does not account for.

Lab scope note: full address mapping may be beyond a day's exercise; still run both, record counts, and state clearly what the check does and does not support.

What it means — modules = file-named images; driverscan = \Driver\ objects. Meaningful join is by address range.

Why it matters on exam — Exam trap: “counts differ $\Rightarrow$ rootkit” is wrong; “orphan Start address” is the real signal.

Example — Driver Start 0xFFFFF80001234000 not inside any modules Base..Base+Size $\rightarrow$ investigate unaccounted kernel code.

19 — What modules/driverscan showed

What it showed: the kernel module list and the driver-object table were both listed and recorded, 141 and 117 records, and neither contains anything that stands out when you look at it. That is a documented look at kernel space, and it belongs in the report as exactly that.

What it did not show: that the kernel is clean. Running these two plugins only *lists* what each one sees. It is not a rootkit test. A real test maps each driver object's **Start** address into a loaded module's **Base .. Base + Size** range and looks for an object that belongs to no loaded image - that is beyond today's scope, and without it you cannot say the kernel was verified.

What modules/driverscan showed vs did not (listing $\neq$ clean)

What it means — Getting a listing only proves you ran the plugins — not that the host is clean.

Why it matters on exam — State limitations: without address mapping you cannot claim “no hidden drivers.”

Example — Lab write-up: “Ran modules + driverscan; counts recorded; address map deferred — listing alone $\neq$ clean.”

20 — vadinfo: hollowing vs reflective

Two columns do the work, not one.

`windows.vadinfo` lists a process's memory regions and, for each one, whether it is **backed by a file on disk**. That separates **file-backed code** from **private code**.

It does **not** separate **process hollowing** from **reflective loading**, because **both are private**. For that you also check whether the **main image region is still intact**.

Read both before you name a technique.

What it means — Private VAD = not mapped from a file. Hollowing and reflective both present as private; check if the main image VAD is still healthy.

Why it matters on exam — Don't name "hollowing" from "private" alone.

Example — Private RWX region + intact file-backed main image → lean reflective/injected; main image region replaced/empty while process name looks legit → lean hollowing hypothesis.

21 — Injection vs hollowing vs reflective

Technique	What the attacker does	How it looks in memory
DLL injection	Forces a legitimate process to load a malicious <code>.dll</code> from disk.	An unexpected module in <code>dlllist</code> , backed by a file - often from a non-standard path such as <code>%TEMP%</code> . The DLL exists on disk, so disk forensics can find it too.
Process hollowing	Starts a legitimate program suspended, unmaps its real code, writes malicious code into the same address space, and resumes it.	The process keeps a trusted name and path , but its main image region is private rather than file-backed , or its in-memory contents do not match the file on disk. Often a suspiciously <i>short</i> module list and no user parent.
Reflective loading	Maps a complete executable into memory itself, bypassing the Windows loader entirely. Nothing is written to disk.	A private RWX region containing an MZ header , belonging to no module in <code>dlllist</code> at all - the executable is invisible to the loader, so it appears in no module list.

None of the three is the same as a DLL being invoked on purpose. Each row above describes code arriving in a process that did not ask for it. On this host you will also meet a DLL that was loaded because a program was *explicitly told* to load it, on its command line. That is not injection, and it is not side-loading either: **side-loading** means planting a DLL where a legitimate program will pick it up *by name*, through its own search order, without being told. Keep the two apart when you report, because they need different detections and they say different things about the attacker.

DLL injection vs hollowing vs reflective; ≠ side-load

What it means — Injection: foreign code into a living process. Hollowing: replace contents of a started process. Reflective: load PE from memory without normal loader. Side-load: drop a DLL where a trusted app will load it from disk search order.

Why it matters on exam — Side-load is a **disk/search-order** story; the others are primarily **memory** stories.

Example — File-backed suspicious DLL next to a signed EXE → side-load lead; private MZ in memory with intact main image → reflective/inject lead.

22 — malfind dump vs dumpfiles

Different technique, different recovery method. The Exercise 4 implant lived in a private memory region, so `malfind --dump` carved it out. This component is a real file that was dropped on disk and started, so `malfind` has nothing to give you and `windows.dumpfiles` is the plugin that does.

Private region → `malfind --dump`; on-disk → `dumpfiles` (forensic extraction plugins)

What it means — Use `malfind --dump` to carve private injected regions; use `dumpfiles` when the image is (or was) file-backed and still referenced in memory.

Why it matters on exam — Wrong plugin = empty or wrong artifact for the question.

Example — Exam: “extract the injected PE with no path on disk” → `malfind dump`; “recover the DLL still mapped from System32” → `dumpfiles`.

F NTFS / MFTECmd

23 — NTFS 0x10 vs 0x30 (timestamp lead)

Every NTFS file carries two sets of timestamps. The `0x10` set is what Windows shows in Explorer, and a program can rewrite it. The `0x30` set is written by the kernel when a file is created, renamed or moved, and ordinary tools cannot set it. Changing the `0x10` set to hide when a file arrived is called **timestamping**, and it leaves a pattern: a `0x10` created time earlier than the `0x30` one.

Copying a file can leave that same pattern, because a copy can inherit the metadata of its source. So treat the pattern as a reason to look, not as a finding on its own. The arrival time you can defend is the `0x30` value, checked against the request and process records you already have.

NTFS 0x10 vs 0x30; timestamp pattern is a lead

What it means — Attribute `0x10` (STANDARD_INFORMATION / SI) is what Explorer-style stamps often show; `0x30` (FILE_NAME / FN) is updated differently and is harder for simple stomps to keep consistent.

Why it matters on exam — SI < FN or other mismatches are a **timestamp lead**, not proof by themselves — correlate with \$Us / Prefetch / logs.

Example — SI Created far earlier than FN Created on the same record → flag timestamp lead; confirm with USN and parent folder times.

24 — MFTECmd

Eric Zimmerman MFT parser — used for `$MFT` / `$J` (USN) timelines, create/modify/delete, and checking **0x10 vs 0x30** timestamps.

Typical lab path:

```
C:\Training\Tools\EricZimmerman\MFTECmd.exe
```

Pairs with: Timeline Explorer, AmcacheParser, Hayabusa.

What it means — Parses MFT/USN into CSV for timeline tools; exposes SI vs FN columns for stomping hunts.

Why it matters on exam — Know which tool answers “0x10 vs 0x30” questions (MFTECmd + Timeline Explorer).

Example — Export \$MFT with MFTECmd → open in Timeline Explorer → filter SI/FN mismatches around the incident window.

Quick exam anchors

- Static order: hash → `file` → entropy/imphash → imports → sandbox lineage
- Dirty hive → collect hive **and** transaction logs
- Native dump EXE → no PS 4104; hunt **4688 / Sysmon 1** cmdline IOCs
- DCSync remediation → domain-wide reset + **krbtgt ×2**
- `pslist` ≠ `psscan`; malfind = leads (RWX+MZ+odd process)
- modules vs driverscan → map by **address**, not name/count
- vadinfo: private vs file-backed; both hollow + reflective are private
- Timestamp lead: SI (`0x10`) vs FN (`0x30`) mismatch